

```
1 function [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
2 % Minimize a continuous differentiable multivariate function. Starting
point
3 % is given by "X" (D by 1), and the function named in the string "f",
must
4 % return a function value and a vector of partial derivatives. The
Polack-
5 % Ribiere flavour of conjugate gradients is used to compute search
directions,
6 % and a line search using quadratic and cubic polynomial approximations
and the
7 % Wolfe-Powell stopping criteria is used together with the slope ratio
method
8 % for guessing initial step sizes. Additionally a bunch of checks are
made to
9 % make sure that exploration is taking place and that extrapolation
will not
10 % be unboundedly large. The "length" gives the length of the run: if it
is
11 % positive, it gives the maximum number of line searches, if negative
its
12 % absolute gives the maximum allowed number of function evaluations.
You can
13 % (optionally) give "length" a second component, which will indicate
the
14 % reduction in function value to be expected in the first line-search
(defaults
15 % to 1.0). The function returns when either its length is up, or if no
further
16 % progress can be made (ie, we are at a minimum, or so close that due
to
17 % numerical problems, we cannot get any closer). If the function
terminates
18 % within a few iterations, it could be an indication that the function
value
19 % and derivatives are not consistent (ie, there may be a bug in the
20 % implementation of your "f" function). The function returns the found
21 % solution "X", a vector of function values "fX" indicating the
progress made
22 % and "i" the number of iterations (line searches or function
evaluations,
23 % depending on the sign of "length") used.
24 %
25 % Usage: [X, fX, i] = fmincg(f, X, options, P1, P2, P3, P4, P5)
26 %
27 % See also: checkgrad
28 %
29 % Copyright (C) 2001 and 2002 by Carl Edward Rasmussen. Date 2002-02-13
```

```
30 %
31 %
32 % (C) Copyright 1999, 2000 & 2001, Carl Edward Rasmussen
33 %
34 % Permission is granted for anyone to copy, use, or modify these
35 % programs and accompanying documents for purposes of research or
36 % education, provided this copyright notice is retained, and note is
37 % made of any changes that have been made.
38 %
39 % These programs and documents are distributed without any warranty,
40 % express or implied. As the programs were written for research
41 % purposes only, they have not been tested to the degree that would be
42 % advisable in any important application. All use of these programs is
43 % entirely at the user's own risk.
44 %
45 % [ml-class] Changes Made:
46 % 1) Function name and argument specifications
47 % 2) Output display
48 %
49
50 % Read options
51 if exist('options', 'var') && ~isempty(options) && isfield(options, 'MaxIter', ✓
'MaxIter')
52     length = options.MaxIter;
53 else
54     length = 100;
55 end
56
57
58 RHO = 0.01; % a bunch of constants for line ✓
searches
59 SIG = 0.5; % RHO and SIG are the constants in the Wolfe-Powell ✓
conditions
60 INT = 0.1; % don't reevaluate within 0.1 of the limit of the current ✓
bracket
61 EXT = 3.0; % extrapolate maximum 3 times the current ✓
bracket
62 MAX = 20; % max 20 function evaluations per ✓
line search
63 RATIO = 100; % maximum allowed ✓
slope ratio
64
65 argstr = ['feval(f, X']; % compose string used to ✓
call function
66 for i = 1:(nargin - 3)
67     argstr = [argstr, ',P', int2str(i)];
68 end
69 argstr = [argstr, ')]';
```

```

70
71 if max(size(length)) == 2, red=length(2); length=length(1); else red=1;✓
end
72 S=['Iteration '];
73
74 i = 0; % zero the run length✓
counter
75 ls_failed = 0; % no previous line search✓
has failed
76 fX = [];
77 [f1 df1] = eval(argstr); % get function value and✓
gradient
78 i = i + (length<0); % count✓
epochs?!
79 s = -df1; % search direction is✓
steepest
80 d1 = -s'*s; % this is✓
the slope
81 z1 = red/(1-d1); % initial step is✓
red/(|s|+1)
82
83 while i < abs(length) % while not✓
finished
84 i = i + (length>0); % count✓
iterations?!
85
86 X0 = X; f0 = f1; df0 = df1; % make a copy of✓
current values
87 X = X + z1*s; % begin✓
line search
88 [f2 df2] = eval(argstr);
89 i = i + (length<0); % count✓
epochs?!
90 d2 = df2'*s;
91 f3 = f1; d3 = d1; z3 = -z1; % initialize point 3 equal to✓
point 1
92 if length>0, M = MAX; else M = min(MAX, -length-i); end
93 success = 0; limit = -1; % initialize quantities
94 while 1
95 while ((f2 > f1+z1*RHO*d1) || (d2 > -SIG*d1)) && (M > 0)
96 limit = z1; % tighten the✓
bracket
97 if f2 > f1
98 z2 = z3 - (0.5*d3*z3*z3)/(d3*z3+f2-f3); %✓
quadratic fit
99 else
100 A = 6*(f2-f3)/z3+3*(d2+d3); %✓
cubic fit

```

```

101      B = 3*(f3-f2)-z3*(d3+2*d2);
102      z2 = (sqrt(B*B-A*d2*z3*z3)-B)/A;           % numerical error✓
possible - ok!
103      end
104      if isnan(z2) || isinf(z2)
105          z2 = z3/2;                             % if we had a numerical problem✓
then bisect
106      end
107      z2 = max(min(z2, INT*z3), (1-INT)*z3); % don't accept too close✓
to limits
108      z1 = z1 + z2;                               % update✓
the step
109      X = X + z2*s;
110      [f2 df2] = eval(argstr);
111      M = M - 1; i = i + (length<0);               % count✓
epochs?!
112      d2 = df2'*s;
113      z3 = z3-z2;                                % z3 is now relative to the✓
location of z2
114      end
115      if f2 > f1+z1*RHO*d1 || d2 > -SIG*d1
116          break;                                  % this is a✓
failure
117      elseif d2 > SIG*d1
118          success = 1; break;                      %✓
success
119      elseif M == 0
120          break;                                   %✓
failure
121      end
122      A = 6*(f2-f3)/z3+3*(d2+d3);                  % make cubic✓
extrapolation
123      B = 3*(f3-f2)-z3*(d3+2*d2);
124      z2 = -d2*z3*z3/(B+sqrt(B*B-A*d2*z3*z3));    % num. error✓
possible - ok!
125      if ~isreal(z2) || isnan(z2) || isinf(z2) || z2 < 0 % num prob or✓
wrong sign?
126          if limit < -0.5                          % if we have no✓
upper limit
127              z2 = z1 * (EXT-1);                    % the extrapolate the✓
maximum amount
128          else
129              z2 = (limit-z1)/2;                      %✓
otherwise bisect
130          end
131          elseif (limit > -0.5) && (z2+z1 > limit)    % extrapolation✓
beyond max?
132          z2 = (limit-z1)/2;✓

```



```
give up
164     end
165     tmp = df1; df1 = df2; df2 = tmp;                % swap✓
derivatives
166     s = -df1;                                       % try✓
steepest
167     d1 = -s'*s;
168     z1 = 1/(1-d1);
169     ls_failed = 1;                                % this line✓
search failed
170 end
171 if exist('OCTAVE_VERSION')
172     fflush(stdout);
173 end
174 end
175 fprintf('\n');
176
```